HOD Individual Viva Questions

Project: Remote Patient Vital Signs Monitoring System / RPM IoMT Patient Monitoring Node

Repository: `RPMIOMTproject`, branch `main`, HEAD `f67c13a`

Use: a bank of cross-project and individual questions with defensible answers. It is deliberately conservative. Where the repository lacks evidence, the answer says so.

Names: member placeholders `[MEMBER 1 NAME]` … `[MEMBER 5 NAME]` must be replaced after the team confirms the real allocation. Do not claim personal authorship of shared work without evidence.

How to answer

1. Lead with a direct answer, then the source/evidence path.
2. State the limitation if the claim is not physically/clinical verified.
3. Use the evidence tags `[SOURCE]`, `[TEST]`, `[BUILD]`, `[RECORDED HW OBS]`, `[UNVERIFIED HW]`, `[PLAN]`.
4. Never "upgrade" a software claim into a hardware or clinical claim.
5. If you do not know, say so and name the file/function to inspect next.

A. Project and architecture — questions 1–8

1. What problem does this project solve?

It is an engineering prototype for local remote monitoring. An ESP32-S3 collects optical pulse/SpO2 estimates, motion and actual TMP117 sensor temperature, plus SOS/fall state; sends one-line JSON over USB serial; a local Python bridge validates it; a dashboard shows live data; SQLite stores valid history; a per-wearer baseline and deterministic alert summary provide context. It explicitly avoids inventing clinical core temperature or fever.

Evidence: `[SOURCE]` `README.md`, `src/main.cpp`, `telemetry_bridge.py`, `telemetry_store.py`, `assets/dashboard.js`.

2. Is it clinically validated or ready for patients?

No. It is an engineering prototype. There is no clinical study, reference validation, regulatory approval, emergency-response integration or real-patient deployment evidence. The core and fever outputs are intentionally unavailable.

Evidence: [UNVERIFIED HW] / not established; audit readiness statement in CODE_AUDIT_MANIFEST.md.

3. Draw the system architecture.



| exact temperature labels

▼

personal_baseline.py + ai_decision_engine.py

seven-day statistical reference + deterministic rules

4. What sensors are installed, and how many measurement sensors are active?

Installed measurement sensors: MAX30102, MPU-6050/MPU-6500 and TMP117 = **3**. The SSD1306 OLED is a display, not a measurement sensor. sensors.active_max is fixed at 3.

Evidence: [SOURCE] firmware sensors block; README hardware table; bridge validation.

5. What is explicitly not installed?

SHT40 and MLX90614 are not installed. Their optional legacy paths are compiled out with RPM_ENABLE_OPTIONAL_SHT40=0 and RPM_ENABLE_OPTIONAL_MLX90614=0. They are not counted in active, baseline, dashboard tabs or inventory. The bridge rejects a packet that claims either is present.

Evidence: [SOURCE] platformio.ini, README; [TEST] optional-sensor rejection tests.

6. What is the telemetry identity and transport?

Firmware rpm-2.1.0, schema 2, USB serial 115200 baud, one JSON object per line with [JSON] prefix. The bridge default is 127.0.0.1:8080.

Evidence: [SOURCE] outputTelemetry(), telemetry_bridge.py.

7. What are the four temperature concepts, and which are currently produced?

1. **Actual TMP117 sensor temperature** — tmp117_sensor_temperature_c; produced when valid.
2. **Clinical wrist-skin temperature** — wrist_skin_temperature_c; currently null, contact unverified.
3. **Estimated core temperature** — estimated_core_temperature_c; always null/not_available.

4. Fever screening — fever_screening; always not_available.

Only the actual sensor temperature is currently available. The other three are deliberately not inferred.

Evidence: [SOURCE] temperature_safety.py, firmware JSON; [TEST] temperature tests.

8. What is the difference between "the team implemented" and a personal contribution?

"The team implemented" is supportable from the repository. A personal contribution claim needs commit/authorship or team confirmation. Without that, each member should say: "In my allocated area, the team's implementation is X; I am responsible for explaining/defending it," not a personal authorship claim for every line.

Evidence: working tree/HEAD; the documentation's authorship warning.

B. Temperature safety — questions 9–18

9. Why is the actual TMP117 range -55..150 °C while the clinical wrist range is -10..60 °C?

-55..150 °C is the accepted actual-sensor code range. -10..60 °C is a clinical guard that would apply only to a future verified wrist observation. The actual reading may be valid outside the clinical wrist range but must never be promoted to wrist/core.

Evidence: [SOURCE] temperature_safety.py, tmp117_protocol.h; [TEST] range tests.

10. Why not add a fixed offset to convert TMP117 to wrist or core?

A fixed offset is not clinically defensible. Wrist temperature is affected by ambient conditions, blood flow, contact pressure and activity; no validated core reference or transformation exists. The source applies **no** fixed offset and does not convert wrist to core.

Evidence: [SOURCE] policy; README; [TEST] core/fever tests.

11. How is TMP117 raw converted, and what does 0xFFFF mean?

The register is signed 16-bit big-endian; °C = signed code × 0.0078125. 0xFFFF as signed is -1, so -0.0078125 °C; a two-decimal display shows -0.01 °C. It is treated as a stuck-bus/all-ones pattern and rejected, not displayed as a real temperature.

Evidence: [SOURCE] src/tmp117_protocol.h; [TEST] test_tmp117_vectors.py.

12. Why reject 0x0000 and 0xFFFF?

They are canonical all-zeros/all-ones bus patterns that can indicate a released, mis-addressed or incompletely read bus. Accepting them would create a fake near-zero temperature. The diagnostic counts them; the main path rejects them.

Evidence: [SOURCE] protocol/safety/bridge/dashboard; [TEST] stuck-pattern tests.

13. What would a verified clinical wrist value require?

All of:

- wrist_contact_status: "verified";
- valid wrist status;
- non-stuck raw code and timestamp;
- signed conversion within 0.02 °C;
- actual TMP117 status valid and matching actual value/raw/timestamp;
- wrist value within -10..60 °C.

No fixed offset. The current build cannot satisfy the contact condition.

Evidence: [SOURCE] evaluate_temperature_safety(), bridge/dashboard/store; [TEST] verified-wrist tests.

14. What are the current wrist fields?

wrist_contact_status: "unverified", wrist_skin_temperature_c: null, wrist_skin_temperature_status: "unverified", wrist_skin_temperature_raw: null, wrist_skin_temperature_timestamp_ms: null.

Evidence: [SOURCE] firmware JSON; [TEST] accepted packet fixture.

15. What is the exact fever message?

Core temperature unavailable — confirmation required

Use the em dash, not a hyphen, when quoting the required label.

Evidence: [SOURCE] temperature_safety.py; [TEST] exact-message assertions.

16. Why can't you estimate core from wrist temperature?

Because there is no validated core reference/transformation. Wrist is peripheral, affected by ambient/contact/perfusion, and cannot confirm or rule out fever. Core stays null/not_available; suspected fever requires a validated clinical thermometer and clinician assessment.

Evidence: [SOURCE] uncertainty/limitations; [TEST] no-core/no-fever tests.

17. What happens when actual TMP117 is valid but wrist contact is unverified?

The bridge accepts the actual fields if the raw/value/timestamp/range rules pass, while requiring wrist status unverified and null wrist evidence. The dashboard shows **TMP117 Sensor Temperature (°C)** separately and leaves **Wrist Skin Temperature (°C)** blank/unverified. Core/fever remain unavailable.

Evidence: [SOURCE] firmware/bridge/dashboard; [TEST] Node/Python cases.

18. What if a non-valid status carries a non-null value?

It is rejected. For actual sensor: non-valid requires value/raw/timestamp null. For wrist: unverified requires all null; verified-but-invalid requires all null. This prevents stale or contradictory evidence from leaking through.

Evidence: [SOURCE] _validate_temperature(); [TEST] contradictory-status tests.

C. Firmware and hardware — questions 19–27

19. Explain the firmware scheduling in one minute.

setup() starts serial, configures Wire (GPIO 8/9, 50 kHz) and Wire1 (GPIO 17/18, 400 kHz), scans/initializes OLED, MPU, MAX30102 and TMP117, then emits the first JSON. loop() debounces SOS/clears alerts, reads MPU every 20 ms, samples optical with a 150 ms gap, reads TMP117 every 2 s, and once per second refreshes states, retries hot-plug, emits JSON and updates the OLED. SOS/fall can emit immediately; urgent output still refreshes freshness first.

Evidence: [SOURCE] setup(), loop(), outputTelemetry().

20. Why two I2C buses?

The primary bus carries slower devices (OLED, MAX30102, TMP117) at 50 kHz. The IMU uses a dedicated Wire1 at 400 kHz on GPIO 17/18. This isolates faster IMU traffic and reduces contention. Both use a 25 ms Wire timeout.

Evidence: [SOURCE] src/main.cpp.

21. How do SOS and fall candidates work, and what are their limits?

SOS: GPIO 5 active LOW with pull-up; 300 ms debounce; first press sets SOS for 15 s and emits urgent telemetry; second press cancels. Fall: free fall <0.6 g, impact >2.5 g within 60–600 ms, active for 6 s. Both are prototype heuristics; physical behavior and clinical validity are unverified.

Evidence: [SOURCE] loop(), updateMpu(); [UNVERIFIED HW] physical tests.

22. How does the firmware handle stale data?

refreshSensorStatuses() runs before every output, including urgent paths. Staleness thresholds: TMP117 6000 ms, MPU 1500 ms, optical 2000 ms; retries every 3000 ms. Stale values become non-valid/null. The bridge also expires the live view after 5 s.

Evidence: [SOURCE] constants/function; bridge STALE_SECONDS=5.

23. What does the TMP117 raw diagnostic add beyond the main firmware?

An independent PlatformIO project that scans 0x48–0x4B, verifies WHO_AM_I=0x0117, reads/writes configuration, prints writable-mask readback and offset, then logs raw MSB/LSB, signed conversion, drdy, failures and all-ones/all-zeros counts every second, with a 30 s summary. It lets the team confirm/refute the -0.01 °C anomaly without main-firmware ambiguity.

Evidence: [SOURCE] diagnostics/tmp117_raw/; [BUILD] fresh success.

24. Has the TMP117 anomaly been proven fixed?

No. Prior observations include raw 0xFFFF → -0.0078125 °C and diagnostic raw 0x0DA7 → 27.30469 °C, 60 reads/0 failures/0 stuck; address 0x49. These were **not reproduced** in this pass. The main firmware rejects stuck patterns, but the physical cause still needs a raw capture.

Evidence: [RECORDED HW OBS]; status **HARDWARE VERIFICATION REQUIRED**.

25. What do the MAX30102 quality guards check?

Engineering thresholds: mean DC 50000..262143, AC at least 1000, perfusion index at least 0.10, plus library validity and finger presence. Firmware accepts HR 30..240 bpm and SpO2 >0..100%; bridge uses HR 20..250, SpO2 0.1..100. These are not clinical accuracy criteria.

Evidence: [SOURCE] sensor_math.h, finishOpticalWindow(); README.

26. What does MPU calibration actually do?

It estimates gyro bias and a single-orientation accelerometer bias from stationary 80-sample windows (ranges ≤0.15 g, ≤4 dps, magnitude 0.7–1.3 g). It is **not** six-face calibration, not temperature-compensated and not a validated fall detector. MPU ranges are ±8 g / ±250 dps with 4096 LSB/g, 131 LSB/dps.

Evidence: [SOURCE] calibrateMpu(), sensor_math.h.

27. What is the OLED limitation?

The SSD1306 is 128×64 with a limited font, so it uses compact TMP:/Wrist: text and two-line core wording. The exact labels/message are implemented in the dashboard/JSON, not guaranteed character-for-character on the OLED. Physical rendering is unverified.

Evidence: [SOURCE] updateOLED(); [UNVERIFIED HW] visual check.

D. Backend, dashboard and storage — questions 28–37

28. What does the bridge do before accepting a packet?

It frames complete serial lines, suppresses identical repeats within 1.5 s, rejects duplicate JSON keys, removes an optional [JSON] prefix, then _validate_packet() checks schema/device/uptime/sensor booleans/active count/status-value consistency/ranges/timestamps/temperature/fever/IMU. A single failure rejects the whole packet. Only accepted packets update live state, storage and baseline comparison.

Evidence: [SOURCE] serial_reader_thread(), parse_packet(), _validate_packet().

29. Difference between duplicate serial lines and duplicate JSON keys?

DuplicateSuppressor drops identical serial text within 1.5 s. Duplicate JSON keys inside one object are always rejected via object_pairs_hook; this prevents last-key-wins ambiguity. They solve different problems.

Evidence: [SOURCE]; [TEST] duplicate tests.

30. Why reject the entire packet instead of one bad field?

A partially accepted patient packet could combine valid and invalid evidence. Whole-packet rejection is fail-closed; the producer must fix the contract or a schema change must be deliberate. The cost is all fields disappear temporarily, which is safer than displaying a contradiction.

Evidence: [SOURCE] _validate_packet() returns boolean; design discussion.

31. What happens after five seconds without a fresh packet?

snapshot() marks the node disconnected, sets a timeout error, and returns telemetry: null and empty comparisons. A newly opened browser is never shown the stale last packet. This is separate from firmware sensor-staleness thresholds.

Evidence: [SOURCE] snapshot(), set_disconnected(); [TEST] stale-no-replay test.

32. What are the local APIs and security posture?

/api/telemetry, /api/stream (SSE), /api/history, /api/baseline (GET/POST), /api/events, static dashboard assets. Default bind is 127.0.0.1:8080. POST baseline has same-origin/size/connected-device checks. There is no authentication/TLS; it is not network/PHI ready.

Evidence: [SOURCE] TelemetryRequestHandler, main().

33. What is stored in SQLite, and what is deliberately not stored?

Tables: observations, enrollments, events. Observations store valid active/legacy values,

statuses, raw wrist code when valid, alerts, device ID and server recorded_at; invalid/missing values are SQL NULL. Actual TMP117 is **not** stored as the clinical wrist history metric. Retention is 30 days; history uses null-filled buckets; device filtering is enforced.

Evidence: [SOURCE] telemetry_store.py; [TEST] history tests.

34. What are the exact dashboard labels, and how are actual versus wrist shown?

Exact labels: **Wrist Skin Temperature (°C)** for the clinical card and **TMP117 Sensor Temperature (°C)** for the actual sensor. While contact is unverified, the wrist card shows no numeric value; the actual sensor can show a valid number separately. Core shows unavailable; fever shows unavailable with the exact message.

Evidence: [SOURCE] assets/dashboard.js, dashboard_core.js, index.html; [TEST] Node cases.

35. How are history gaps and CSV handled?

History creates a null point for every missing bucket. DashboardCore.gapAwarePath() starts a new M segment at a null, so the chart breaks instead of interpolating. CSV exports active metrics only: timestamp, HR, SpO2, clinical wrist, motion, samples; missing cells are blank, not zero. Actual TMP117 is not exported as wrist history.

Evidence: [SOURCE] telemetry_store.history(), gapAwarePath(), csv(); [TEST] gap/CSV tests.

36. Why does the baseline never become fully ready in the current build?

Overall readiness requires all four active metrics to have models. Wrist values are not stored while contact is unverified, so the wrist model cannot qualify. This is intentional; injecting actual TMP117 into the wrist column would be a safety regression. The system can be collecting/partial, not fully ready.

Evidence: [SOURCE] personal_baseline.py, telemetry_store.measurements(); [TEST] baseline/history tests.

37. What does the dashboard do with a storage error?

The bridge catches sqlite3.Error, records hardware_status['storage_error'], and still updates/broadcasts live telemetry. The dashboard should surface the error; history completeness is not claimed. The live view is not fabricated to hide the failure.

Evidence: [SOURCE] _accept_packet().

E. Baseline, analytics and quality — questions 38–45

38. What is the baseline model and qualification rule?

Seven days, at least 30 distinct sampled minutes per day, minute means, then a frozen per-wearer reference:

center = median(values)

width = max(3 × 1.4826 × MAD, noise floor)

low/high = center ± width

Noise floors: HR 5 bpm, SpO2 2%, wrist 0.2 °C, motion 0.1 g. It is a statistical reference, not a diagnosis.

Evidence: [SOURCE] personal_baseline.py; [TEST] baseline tests.

39. When is a baseline deviation event recorded?

After three consecutive outside readings, each within 5 s of the prior, with at least 300 s since the last event for that metric. It is a review event, not a diagnosis. In-memory run state resets on process restart.

Evidence: [SOURCE] compare().

40. Is the alert engine AI? What are its thresholds?

No. It is deterministic threshold logic. SOS/fall → RED (EMERGENCY); HR <50 or >120, or SpO2 <90 → YELLOW (WARNING); both optical values valid with no threshold → GREEN (NO OPTICAL ALERT); otherwise UNKNOWN (MISSING DATA). There is no trained model/LLM.

Evidence: [SOURCE] ai_decision_engine.py; [TEST] engine tests.

41. Why does missing optical data produce UNKNOWN rather than GREEN?

GREEN means both HR and SpO2 were validated and no configured optical threshold was crossed. If either is unavailable, the engine cannot claim reassurance, so it returns UNKNOWN (MISSING DATA) and states that no value was substituted.

Evidence: [SOURCE] evaluate_clinical_status(); [TEST] missing-data tests.

42. What tests and builds pass, and what do they prove?

Fresh on 2026-10-29: **62 Python tests passed**, **10 Node dashboard tests passed**, Ruff/Python compile/JS syntax passed; main firmware SUCCESS (RAM 21,424/327,680; flash 326,001/6,553,600); TMP117 diagnostic SUCCESS (RAM 18,916/327,680; flash 282,377/6,553,600). They prove host/build behavior, not sensor accuracy, contact, clinical validity or assembled hardware.

Evidence: [TEST]/[BUILD]; no hardware validation.

43. What is the status of prior hardware values?

Prior observations: address 0x49, raw 0x0ED3 → 29.6484375 °C, raw 0xFFFF → -0.0078125 °C, diagnostic 0x0DA7 → 27.30469 °C, 60 reads/0 failures/0 stuck patterns. None were reproduced in this pass. Label them **RECORDED HW OBS — not current validation; HARDWARE VERIFICATION REQUIRED.**

Evidence: CODE_AUDIT_MANIFEST.md; no fresh capture.

44. Which audit items are closed versus open?

C-01–C-14 are closed **in software** (source/tests/builds), including actual/wrist separation, unverified contact, range, raw/stuck handling, duplicate keys, core/fever, legacy aliases, OLED bounds, gaps, actual-as-history, baseline, labels and coverage. O-01–O-07 remain open: TMP117 physical evidence, accuracy, contact method, MAX/MPU, OLED/SOS/end-to-end, clinical/regulatory suitability and CI. Do not say "all audit items are closed."

Evidence: CODE_CLEANUP_REGISTER.md, NEXT_STEPS.md.

45. What are the biggest false claims to avoid?

- TMP117 actual = clinical wrist/core.
- Wrist = core or fever screening.
- Software tests = hardware/clinical validation.
- SHT40/MLX installed/tested.
- The deterministic engine = trained AI/LLM.

- The project = emergency-response/clinical device.
- All audit items closed.
- Personal authorship of every shared file.
- History continuity/zero substitution.

Evidence: all project policy and evidence records.

F. Individual contribution questions — for every member

46. "What exactly did you personally contribute?"

Answer with a confirmed task, the source path/behavior and the evidence — not with "everything."

Safe template:

> "My confirmed allocation is [area]. The team's implementation I am responsible for defending is [specific files/functions]. The evidence is [test/build/source]. I am not claiming authorship of every line; if we do not have authorship records, I describe it as a team implementation."

Use the matrix to choose the area:

Member	**Defensible contribution to name**	**Source/evidence to point to**
[MEMBER 1 NAME]	Firmware integration/scheduling/JSON/ OLED/SOS/fall	src/main.cpp; builds
[MEMBER 2 NAME]	TMP117 protocol/diagnostic; MAX/MPU hardware reliability	tmp117_protocol.h, sensor_math.h, diagnostic; vector/build evidence
[MEMBER 3 NAME]	Serial parser/validation/SSE/APIs	telemetry_bridge.py; telemetry tests
[MEMBER 4 NAME]	Dashboard labels/gaps/CSV; SQLite history	dashboard_core.js, dashboard.js, telemetry_store.py; Node/history tests

Member	Defensible contribution to name	Source/evidence to point to
[MEMBER 5 NAME]	Temperature/baseline/alert policy; audit evidence	temperature_safety.py, personal_baseline.py, ai_decision_engine.py; Python tests/audit docs

47. "Can you prove you wrote that file?"

Only with a commit/team record. The current working tree includes prior uncommitted work that predates this documentation task; this task did not modify production source. If records are unavailable, say so and defend the technical area rather than making an authorship claim.

Evidence: repo status/HEAD; matrix warning.

48. "What would you personally improve first?"

A defensible answer tied to the area:

- Member 1: hardware-in-the-loop timing/recovery tests; bounded I2C recovery; verified OLED layout.
- Member 2: raw TMP117 capture; reference/contact protocols; MAX/MPU validation.
- Member 3: fault-injection/load/security tests; structured rejection reasons.
- Member 4: browser E2E/accessibility/security tests; separate diagnostic history if needed.
- Member 5: contact method; formal accuracy/baseline/alert evaluation; CI/evidence artifact.

Evidence: NEXT_STEPS.md; member documents.

49. "What would you refuse to change, and why?"

Refuse to:

- copy actual TMP117 into clinical wrist/core;
- add a fixed offset without validated evidence;
- make core/fever numbers appear;
- substitute zero/stale values for missing data;
- count SHT40/MLX as installed;

- weaken bridge validation to make a demo pass;
- call the deterministic engine AI/LLM;
- claim clinical/emergency readiness.

Evidence: safety policy, tests, audit red lines.

50. "Give the 30-second version of the project."

"It is an ESP32-S3 prototype that collects optical, motion and actual TMP117 sensor data, streams fail-closed JSON to a local bridge, stores valid history and shows a dashboard. The clinical wrist value stays unverified/null, core is unavailable and fever is not diagnosed. The baseline is statistical and the alert engine is deterministic, not AI. The software tests/builds pass; hardware, accuracy, contact and clinical validity remain unverified."

Evidence: shared understanding; all modules.

G. Rapid-fire answer key

Question	**Short answer**
Firmware version / schema	rpm-2.1.0 / 2
Serial / prefix	115200 / [JSON]
Installed measurement sensors	MAX30102, MPU, TMP117 = 3
Not installed	SHT40, MLX90614
Actual TMP range	-55..150 °C
Wrist range with verified contact	-10..60 °C
Raw LSB	0.0078125 °C
Stuck patterns	0x0000, 0xFFFF
Current wrist	null/unverified
Current core	null/not_available
Current fever	not_available
Exact fever message	**Core temperature unavailable — confirmation required**
Bridge stale timeout	5 s
Duplicate serial line window	1.5 s
Baseline qualification	7 days, ≥30 minutes/day

Question	Short answer
Baseline formula	median ± max(3 × 1.4826 × MAD, floor)
Deviation trigger	3 consecutive outside, 300 s cooldown
Engine thresholds	HR <50/>120; SpO2 <90; SOS/fall red
AI/LLM?	No; deterministic
Tests fresh	62 Python + 10 Node
Main build	SUCCESS, RAM 21,424/327,680; flash 326,001/6,553,600
Diagnostic build	SUCCESS, RAM 18,916/327,680; flash 282,377/6,553,600
Hardware validation	Not performed in this pass

H. If the HOD challenges an answer

1. **"Where is that in the code?"** Open the function named in the answer, not just the README.
2. **"Did you run it?"** Distinguish host tests/builds from hardware. Give exact command/result/date.
3. **"Did you personally write it?"** Use the authorship rule; do not guess.
4. **"Is it clinically valid?"** Answer no/not established; explain the validation gap.
5. **"Why not just show a number?"** Because null is honest; a fabricated core/fever/wrist value is worse than an empty card.
6. **"Why is the wrist blank if TMP117 works?"** Actual sensor ≠ clinical wrist; contact is unverified.
7. **"Why is the baseline not ready?"** Overall readiness needs all four metrics; unverified wrist is intentionally not stored.
8. **"Why call it AI?"** Historical filename only; the code is deterministic. Do not defend a false AI claim.

I. Closing standard

Every answer should leave the examiner with three things:

1. **What the software actually does.**

2. What evidence exists — and what does not.

3. Why the conservative choice is intentional, not an accident.

If those three are clear, the member can defend the project without overclaiming.